

Experiment Report

Name: 李崇楷 Student ID: 313834006

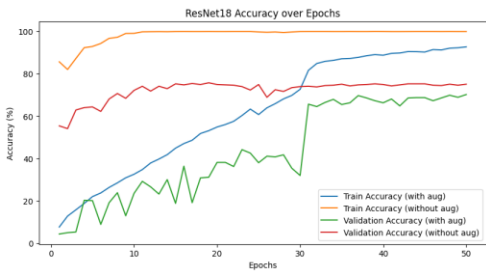
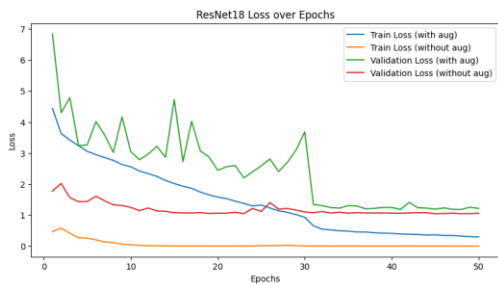
Code: <https://github.com/kailee0422/Deep-Learning.git>

Introduction

In this assignment, the task is to develop a simple ResNet model for image classification using the Flowers102 dataset. This dataset consists of 102 flower categories, and the goal is to accurately classify the images into their respective categories. The dataset is loaded using Flowers102, and since the images vary in size, appropriate resizing methods are applied to standardize the image dimensions for efficient processing by the CNN. The primary objective is to experiment with different CNN architectures and training techniques to identify the model that achieves the highest accuracy in classifying the flower images. Although I had learned some CNN during university, I didn't delve deeply into model training, which led me to make several mistakes. Therefore, I divided my experiments and results into two parts: one for the unsuccessful attempts and the other for the more successful and reasonable outcomes.

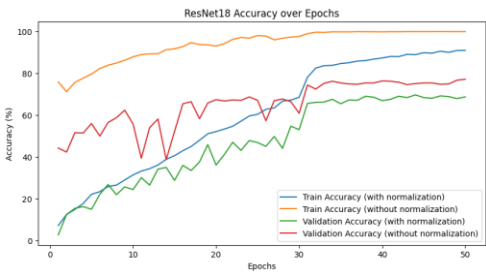
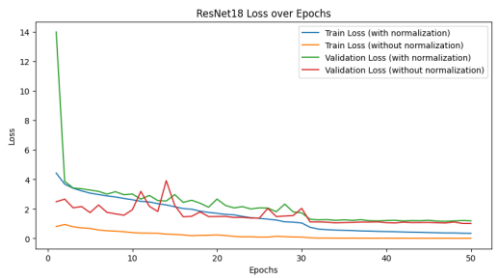
Fail Experiment and Result

Here are some of the mistake I encountered :First, when using the test loader, I mistakenly applied an augmentation to the test data, which is something that typically shouldn't be done. Second, during data augmentation, I chose too few augmentations and didn't include the original images during training. This caused the model to only learn the augmented images and not the features of the original images. Additionally, I wanted to experiment with whether increasing image size would improve the model's accuracy, so I consistently used a batch size of 128 because of the speed. However, I later realized that this approach was somewhat flawed. While training speed did increase, the optimization and generalization were worse compared to using a smaller batch size. Even using Adam as the optimizer didn't help improve the results. After further observations, I realized that simply resizing the images wasn't enough—adjustments to the model's dimensions and other parameters were necessary. I hope not to repeat these mistakes. Below are some of the incorrect results I obtained: I initially conducted a simple test using ResNet18, with and without data augmentation.



Augmentation	Accuracy
Yes	66.47%
No	66.08%

Although the accuracy only improved slightly, I believe that augmentation is very important in image recognition tasks, so I used it in all subsequent experiments. Next, I conducted a simple experiment using ResNet18 to test whether or not to apply normalization to the images. Here I use ImageNet mean and std. The results are as follows:



Normalization	Accuracy
Yes	48.43%
No	2.94%

It can be observed from the results that there is a significant difference in accuracy, so I used ImageNet's mean and standard deviation for normalization.

Next, I attempted to test the model with various image sizes and optimizers but was unaware that my code had an error, which caused me to waste a lot of time training with different image sizes. The image sizes I experimented with were 224x224, 320x320, and 640x640, while the optimizers I tried were the original SGD from the research paper and the more commonly used Adam optimizer. Below are my results. Since I believe that including the loss and accuracy graphs would not contribute positively to the report, as they represent unsuccessful attempts, I have only included the accuracy results.

Image Size Model Opti: SGD	Accuracy	Image Size Model Opti: Adam	Accuracy
ResNet18 224 x 224	65.88%	ResNet18 224 x 224	73.14%
ResNet18 320 x 320	73.63%	ResNet18 320 x 320	72.16%
ResNet18 640 x 640	72.94%	ResNet18 640 x 640	69.61%
ResNet34 224 x 224	61.86%	ResNet34 224 x 224	74.71%
ResNet34 320 x 320	72.16%	ResNet34 320 x 320	71.86%
ResNet34 640 x 640	-	ResNet34 640 x 640	69.71%
ResNet50 224 x 224	44.12%	ResNet50 224 x 224	76.37%
ResNet50 320 x 320	60.75%	ResNet50 320 x 320	74.02%
ResNet50 640 x 640	-	ResNet50 640 x 640	-

Note: "- means because I have run many program at the same time in VS code. My VRAM is full, but I have no time to try and I found my problem, so I don't experiment continuously.

Additionally, because I didn't realize that the Flower102 dataset already includes a validation set, I used a validation ratio setting to split the training data. In the subsequent experiments, I made sure to use the Flower102 validation set for evaluation. To summarize, here are the mistakes I made so far:

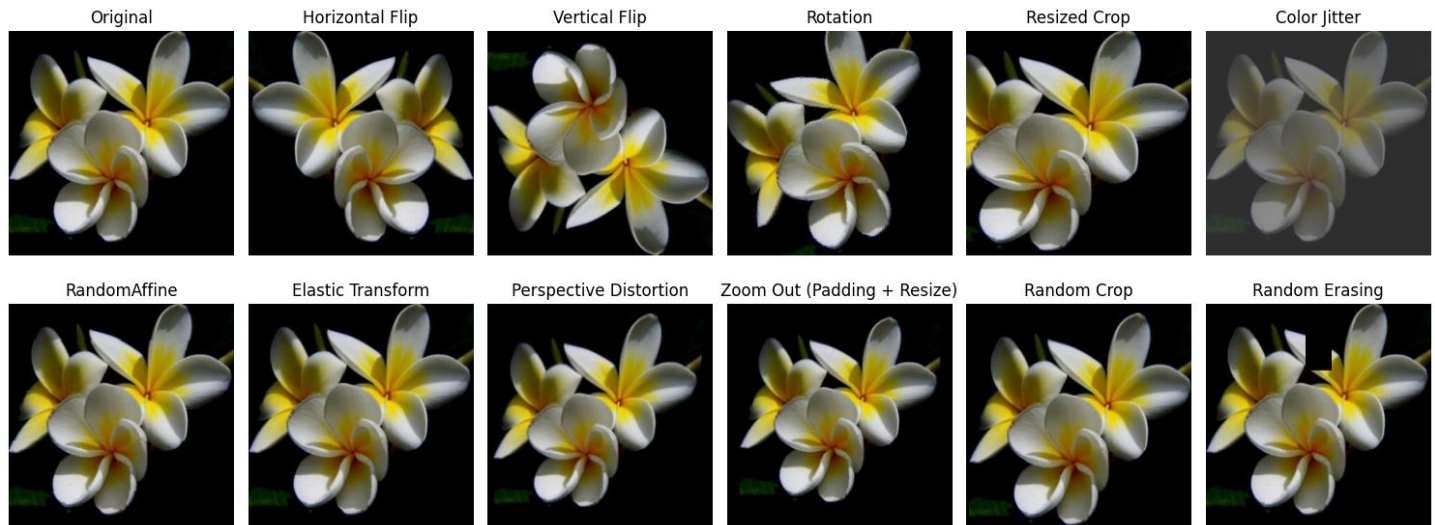
- Batch size was too large.
- Data augmentation was insufficient.
- I experimented with various image sizes, but they had no effect on the model because the code had some mistakes.
- The Flower102 dataset has a validation set, so there's no need to manually split it.

Therefore, in the successful examples, I addressed and improved upon the above-mentioned issues.

Successful Experiment and Result

Adjustment 1: I originally set the batch size to 128, but it was too large. Therefore, I set the batch size to 32 for the upcoming experiments.

Adjustment 2: In my initial experiments, I only used RandomVerticalFlip, RandomHorizontalFlip, and RandomRotation. However, this was far from sufficient for the flower102 dataset, so I adopted the following methods and let my train dataset twice.



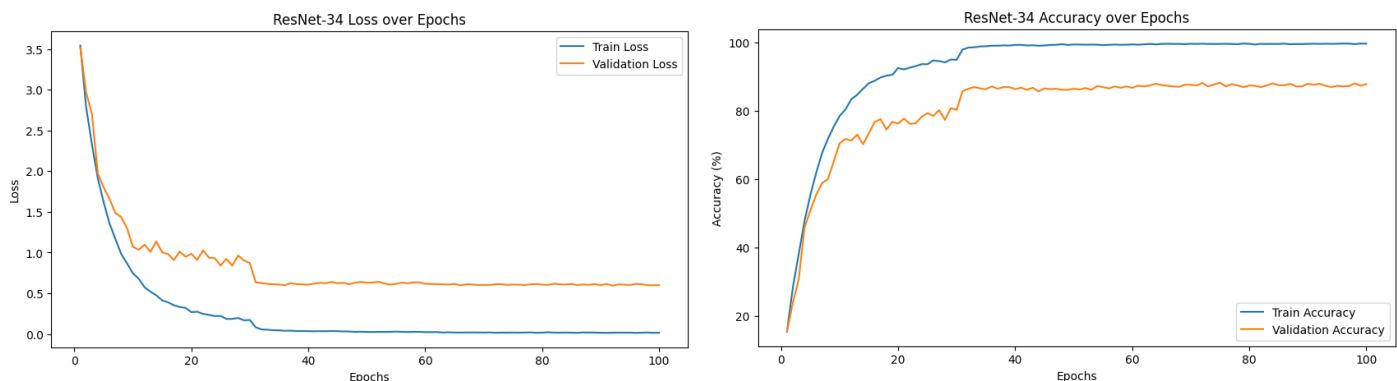
Adjustment 3 and Adjustment 4: I only need to change my resize parameter and validation dataloader.

First, I conducted tests using five different optimizers: SGD (from the original paper), Adagrad, RMSprop, Adam, and AdamW. The following are the accuracy results.

	SGD	Adagrad	RMSprop	Adam	AdamW
ResNet18	83.33%	75.78%	79.90%	83.43%	85.39%
ResNet34	80.69%	77.55*%	77.65*%	80.78%	88.04%
ResNet50	81.67%	-	-	82.55%	85.59%

Note: * means My loader use num_worker, so my CPU use 100% and lead my lab's UBS alarming. I reboot my computer , but I didn't save my data. So I only have epoch 90's test accuracy. – means I think it's not necessary to test.

Below are the charts showing the loss and accuracy when using AdamW and the StepLR scheduler.

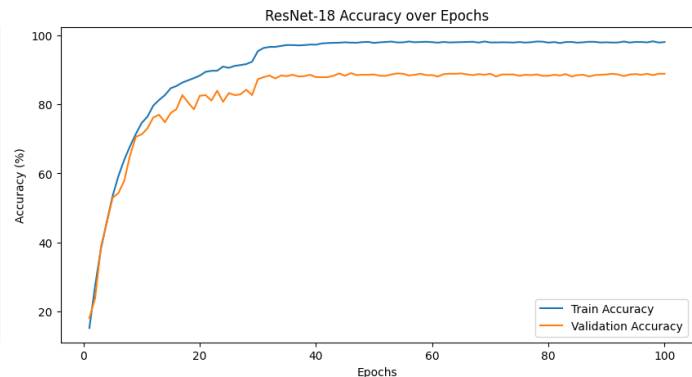
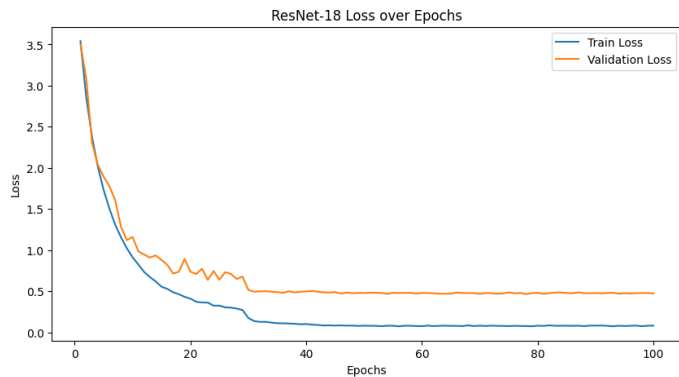


Next, I adjusted the scheduler for AdamW and observed the changes in accuracy.

	StepLR	CosineAnnealingLR	ReduceLROnPlateau
ResNet18	85.39%	83.73%	89.22%
ResNet34	88.04%	-	88.63%
ResNet50	85.59%	-	90.98%

Note: – means I have no enough HW and time to test 😞

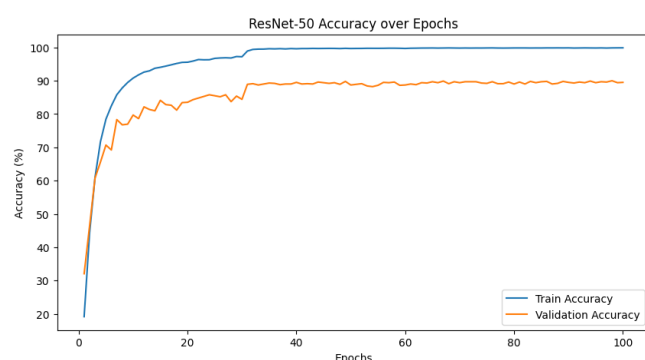
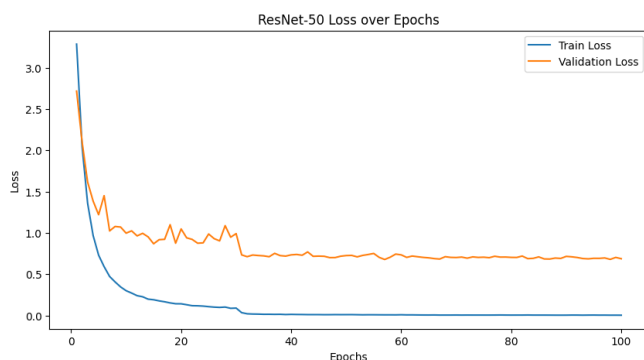
Below are the charts showing the loss and accuracy when using AdamW and the ReduceLROnPlateau scheduler.



In addition, since the number of samples in each category of the flower102 dataset is imbalanced, I conducted a test to address this issue. I first identified the category with the largest sample size and used its count plus an additional 100 as the benchmark. Then, I augmented the number of samples in each category to match this benchmark. This ensures that categories with more samples do not perform better, while those with fewer samples do not perform worse.

	Without Balance	Balance
ResNet18	85.39%	86.67%
ResNet34	88.04%	89.02%
ResNet50	85.59%	88.63%

Below are the charts showing the loss and accuracy when using Resnet50 balance num of each class , AdamW optimizer and StepLR scheduler.



Based on the results, it was observed that using ResNet50 with the AdamW optimizer and the ReduceLROnPlateau scheduler achieved an accuracy of 90.98%. Due to a requirement in the assignment to adjust the activation function and observe the changes in accuracy, I tested the original ReLU from the paper, along with variations like LReLU, EReLU, and PReLU. Below are the accuracy results for each activation function.

	ReLU	LReLU	EReLU	PReLU
ResNet50 Optim:AdamW ReduceLROnPlateau	90.98%	90.20%	89.51%	89.41%

It was observed that using the ReLU activation function, as provided in the original paper, resulted in the highest accuracy. Therefore, redundant images are not included below.

Next, I conducted two small experiments. One focused on balancing the number of samples in each class. In the original method, I used the AdamW optimizer and the stepLR scheduler. However, based on the previous results, I noticed that using the ReduceLROnPlateau scheduler performed better than the stepLR scheduler. Therefore, I replaced the scheduler and compared the results with the original approach. Below are the comparison results.

	Balance stepLR	Balance ReduceLROnPlateau
ResNet50	88.63%	86.96%

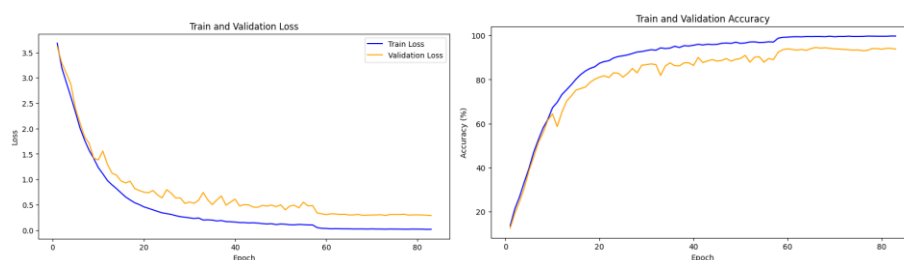
Note: Due to time constraints, I was unable to test ResNet18 and ResNet34, so I only used ResNet50 for this experiment.

The final experiment focused on resizing the image input size for ResNet18, ResNet34, and ResNet50 models using the AdamW optimizer and the ReduceLROnPlateau scheduler. The original image size of 224 x 224 was increased to 512 x 512. Although this adjustment extended both computation time and memory usage, a larger image size allows for better preservation of the flower's features. The following are the results of the experiment:

	224 x 224	512 x 512
ResNet18	89.22%	89.71%
ResNet34	88.63%	93.04*%
ResNet50	90.98%	91.76*%

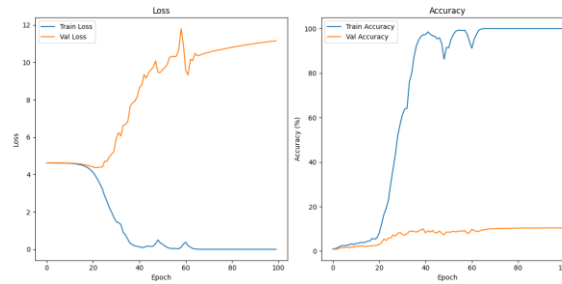
Note:* indicates that I only tested 80 epochs because I accidentally closed it.

The chart below was generated in Colab, but I accidentally closed it, so it didn't successfully generate the chart. Additionally, I don't have time to let it run again.



Implement:

In the above experiment, it was found that the highest accuracy using ResNet was 93.04%, but when using the sample code (CNN), the accuracy was only 10.2%. The chart below shows the train/val loss and train/val accuracy for the sample code.



Discussion

1. Why does the sample code perform worse than ResNet?

The performance of CNNs is generally worse than that of ResNet, especially with two-convolution-layer structure, which is prone to the vanishing gradient problem as the network depth increases, limiting its learning capability. Additionally, overfitting can easily occur after 25 epochs based on above chart, while ResNet's skip connections effectively mitigate these issues. Therefore, in terms of deep network architecture and generalization performance, ResNet typically outperforms traditional CNNs.

2. Why is ReLU better than LReLU, EReLU, and PReLU?

ReLU may perform better than LReLU, EReLU, and PReLU in the Flower102 dataset with ResNet for several reasons. Firstly, ReLU is simple and computationally efficient, allowing for faster convergence during training. Secondly, it promotes sparsity by setting negative values to zero, which can enhance feature selection and model performance. Thirdly, ReLU helps maintain stronger gradients during backpropagation, reducing the risk of the vanishing gradient problem. Lastly, its empirical success in deep networks like ResNet makes it a preferred activation function for many tasks, leading to consistently strong performance in various datasets.

Conclusion

According to the experimental results, from the perspective of model architecture, ResNet18, ResNet34, and ResNet50 are structurally similar; however, the choices of data preprocessing, optimizer, and learning rate scheduler significantly impact accuracy. In terms of data preprocessing, we experimented with various methods, including data transformations, class balance, and image size adjustments. The results indicate that applying data transformations leads to a significant improvement in model accuracy. Class balancing also results in a certain degree of accuracy enhancement. However, when utilizing optimizers that perform better with other models, the accuracy may decrease. Additionally, the high number of augmentations increases the required hardware resources, thus it is recommended to ensure that the CPU performance is sufficiently powerful when training with class balancing. Regarding the impact of image size, the experimental results show that larger image sizes correlate with higher accuracy, which can be attributed to the AdaptiveAvgPool2d layer used in this experiment. Even though the input to the fully connected layer remains unchanged after this layer, larger images are better at preserving image features. In terms of optimizer selection, the commonly used Adam and AdamW have shown excellent performance. As for the learning rate scheduler, ReduceLROnPlateau exhibited the best performance; however, in cases with larger training datasets, switching to this method led to a decrease in accuracy.

Acknowledgment

Thanks to everyone at 206 Lab and the seniors from other lab. I hope this project will meet the assistant's expectations.